

Network Intrusion Detection on NSL-KDD

A Critical Reproduction Study

Data Science in Cyber · Final Project · Intrusion Detection Systems

Source evaluated: *Mamcose* — *NSL-KDD Network Intrusion Detection*

Executive Summary

This project critically evaluates a widely referenced tutorial that trains machine-learning classifiers on the NSL-KDD benchmark to detect network intrusions. The source loads the official train and test files, splits attacks into the four standard families (DoS, Probe, R2L, U2R), trains Random Forest, K-Nearest-Neighbours and a linear SVM, evaluates them with 10-fold cross-validation, and reports accuracies up to ~99.8%, concluding that the approach detects intrusions almost perfectly. We reproduced the pipeline and tested whether that conclusion holds.

The near-perfect accuracy is an evaluation artefact. NSL-KDD ships a dedicated test set, KDDTest+, that deliberately contains **17 attack types never seen in training** (16.6% of the test set). Cross-validating inside one distribution, as the source does, removes this challenge. When we use the protocol the benchmark was built for (train on KDDTrain+, test on KDDTest+), our best model (XGBoost) drops from 99.9% to **79.5%** accuracy, an MCC of only 0.6427, and the severe rare attacks collapse: **R2L recall 6.3%, U2R recall 19.4%**, and novel-attack recall 38.7%.

In short, the headline number measures interpolation within a single 25-year-old distribution, not the generalisation to unseen attacks that an intrusion-detection system requires. The 'Random Forest is best' ordering holds, but the near-perfect accuracy should not be cited as evidence of a deployable detector.

1. Summary of the Source

Problem and why it matters

A network intrusion-detection system (IDS) flags malicious traffic — denial of service, scanning, remote intrusions, privilege escalation — among a large volume of benign connections. It is a classic, high-stakes classification problem: missed attacks are breaches, and excessive false alarms exhaust the analysts who must triage them.

Dataset, solution and methodology

The source uses **NSL-KDD**, the de-duplicated successor to KDD'99: 125,973 training and 22,544 test connection records, each with 41 features (three categorical: protocol, service, flag; the rest numeric counts, rates and byte totals) plus a label. It maps the raw labels to four attack families, trains Random Forest, KNN and a linear SVM, and measures accuracy by 10-fold cross-validation, reporting Random Forest highest at roughly 99.8%.

2. Critical Evaluation of the Source's Claims

We confront each claim with evidence from our reproduction.

Source's claim	Our evidence	Verdict
~99.8% accuracy means a near-perfect IDS	Reproduced under same-distribution CV (99.9%), but accuracy falls to 79.5% on the official KDDTest+ (a 20.4% drop)	Not supported
Accuracy is an adequate metric	R2L recall 6.3% and U2R recall 19.4% are far below the headline; MCC is only 0.6427	Not supported
The model detects attacks	It detects seen patterns, but recall on the 17 novel attack types is 38.7%	Partially supported
The pipeline is sound	Feeds a constant feature, ignores one-hot train/test alignment, never tests novel attacks	Weak methodology
Random Forest is the best model	Confirmed: tree ensembles lead under both protocols	Supported

2.1 The evaluation protocol, not the model, drives the headline

NSL-KDD's test set is built to contain attack types absent from training, so a model must **generalise to novel attacks**. Cross-validating inside KDDTrain+ measures something easier: recognising variants of patterns already seen. We trained six models under both protocols. Under same-distribution CV every model scores in the high 0.99s, reproducing the source. Under the official train→test protocol, accuracy falls by about twenty points (Figure 1).

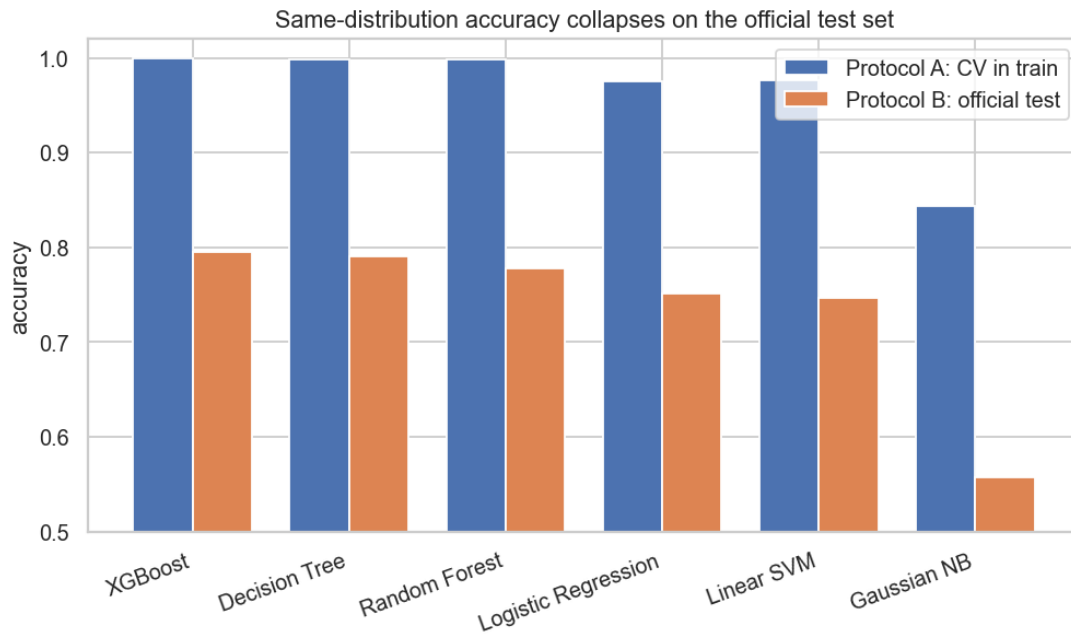


Figure 1. Same-distribution cross-validation accuracy (blue) versus official KDDTest+ accuracy (orange). The ~20-point gap is the difference between recognising seen attacks and detecting novel ones.

2.2 Accuracy hides the dangerous attacks

Two of the four attack families, R2L (remote-to-local) and U2R (user-to-root), are under 1% of training yet are the most severe. The per-family detection rates on the official test set (Figure 2) show high-volume DoS and Probe attacks caught well, but R2L and U2R recall far lower, and the 17 novel signatures missed at a high rate. A single accuracy figure, dominated by easy traffic, conceals exactly the failures a defender most cares about.

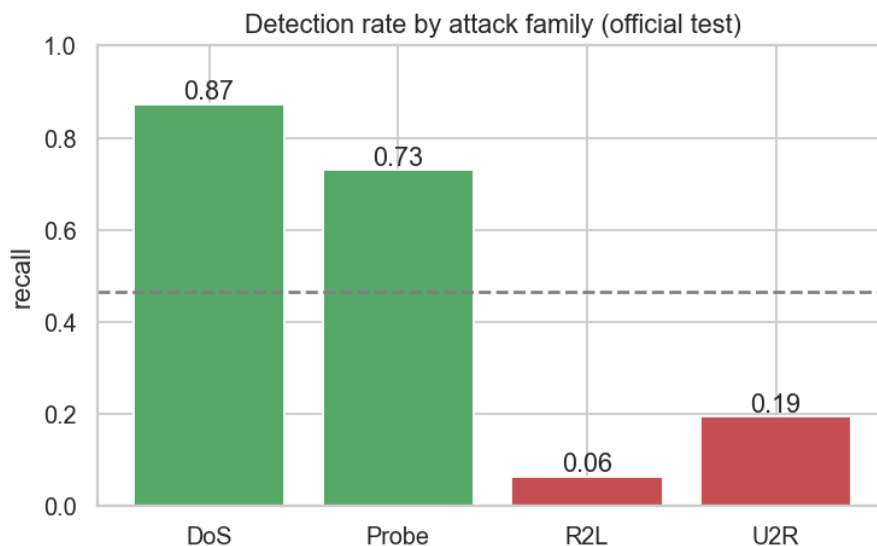


Figure 2. Detection rate (recall) by attack family on the official test set; families below 50% recall are shown in red. The severe, rare R2L/U2R families are detected far less reliably than DoS/Probe.

2.3 Methodology gaps

- A constant feature, `num_outbound_cmds` (all zeros in training), is fed to every model.
- Evaluation is same-distribution cross-validation, which removes the novel-attack challenge the benchmark was created to pose.
- One-hot encoding is not aligned between train and test, a silent reproducibility trap.

- Accuracy is the only reported metric; MCC, per-class recall and the false-positive / false-negative trade-off are absent.
- A 2009 dataset modelling 1998-1999 traffic is treated as representative of modern networks.

Are the conclusions justified? 'Random Forest performs best here' is justified. 'Machine learning detects network intrusions almost perfectly' is **not**: the figure is produced by an evaluation that avoids the benchmark's central difficulty.

3. Feature Engineering Analysis

Was feature engineering performed? The source does minimal preprocessing. We add the steps the data demands. The byte and duration counts are extremely right-tailed (src_bytes spans zero to over a billion), so we apply a **$\log(1+x)$ transform** to compress the tail into a usable, near-unimodal shape; without it, distance- and gradient-based models are dominated by a few bulk transfers (Figure 3).

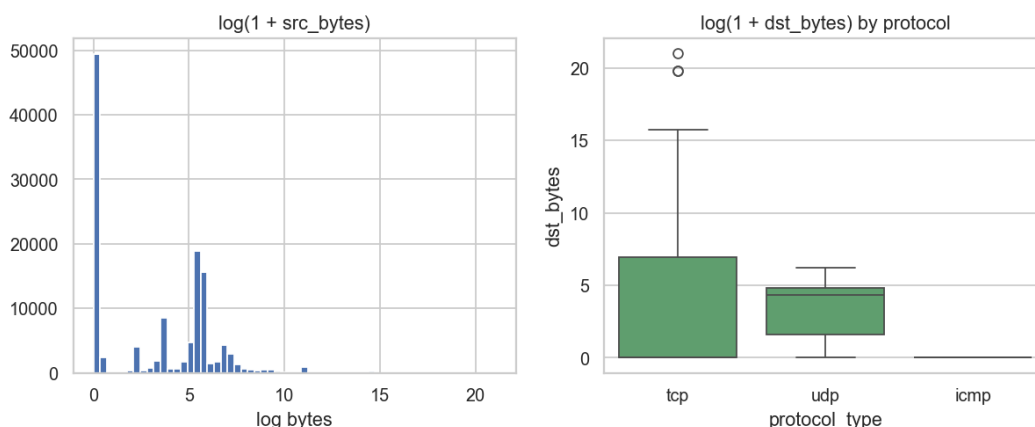


Figure 3. Byte-volume features after a $\log(1+x)$ transform. The raw scale spans nine orders of magnitude; the transform is mathematically necessary, not cosmetic.

Encoding, scaling, redundancy and selection

- **Encoding:** the three nominal features (protocol, service, flag) are **one-hot encoded** rather than integer-labelled, which would invent a false ordering. The one-hot space is fixed on training and the test set re-indexed onto it so the two stay aligned.
- **Scaling:** linear, naive-Bayes and SVM models are pipelined with StandardScaler (fit on training folds only); tree ensembles are scale-invariant and need none.
- **Redundancy:** whole blocks of features are near-perfectly correlated (the serror group, the dst_host host-based aggregates). We detect this with a Spearman correlation matrix (Figure 4) and rely on mutual-information ranking and tree ensembles rather than feeding all features in blindly.
- **Selection:** mutual information confirms that a modest subset (flag, service, count and host-based rates) carries most of the signal.

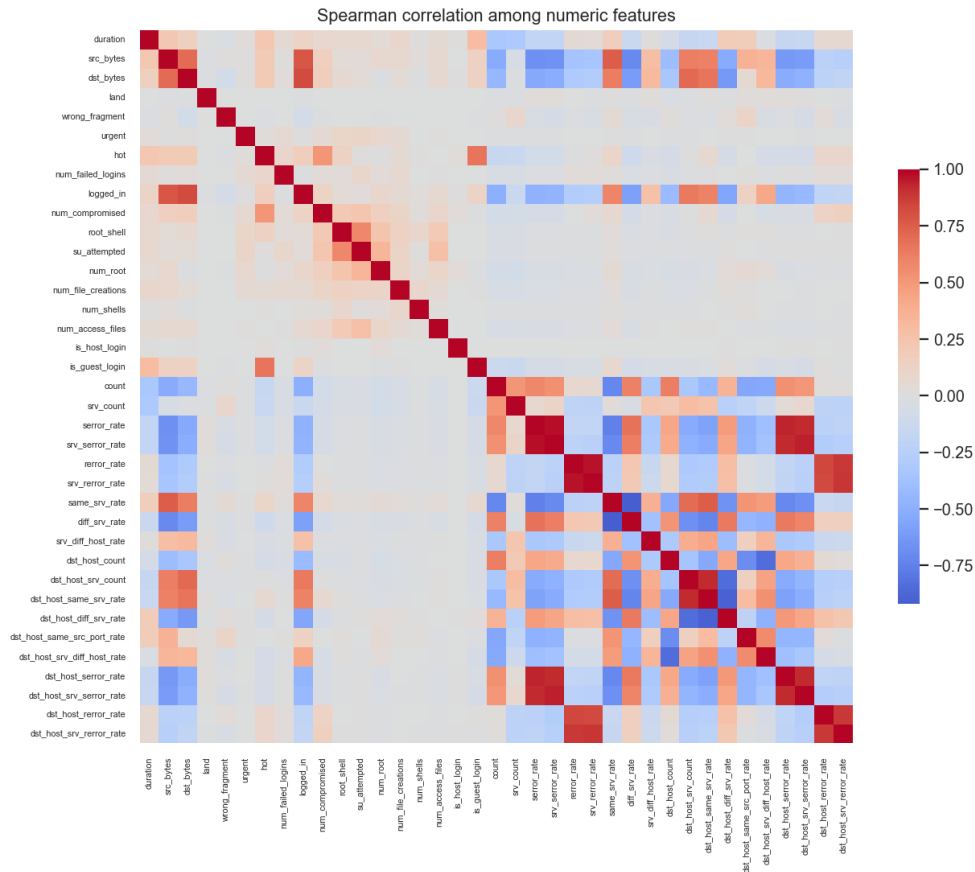


Figure 4. Spearman correlation among numeric features (chosen over Pearson because the features are skewed, non-linear counts and rates). Bright off-diagonal blocks indicate redundancy.

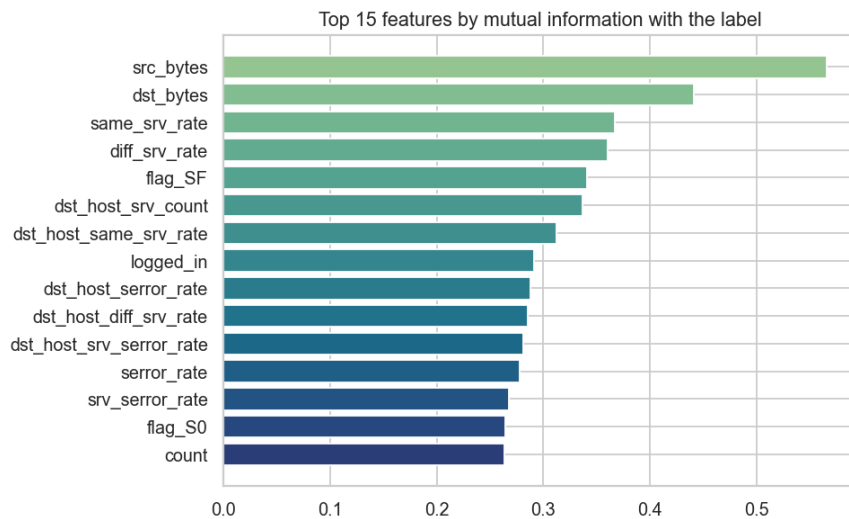


Figure 5. Top features by mutual information with the label, used for transparent feature selection that preserves the original meaning of each feature.

Additional features that could help

- Time-window aggregates from raw flows (connections per host per minute) for modern captures.
- Payload/entropy features to separate stealthy R2L/U2R sessions from benign logins.
- Cost-sensitive class weights or resampling to lift the rare-but-severe families.

4. Reproducibility Analysis

- **Can the code run?** Yes; given the public NSL-KDD files the pipeline executes end-to-end (we reproduced it).
- **Hidden preprocessing:** the trailing difficulty column is a builder artefact that leaks the answer if used as a feature; the constant num_outbound_cmds adds noise. Neither is flagged by the source.
- **The reproducibility trap:** one-hot encoding train and test independently yields different column sets; without explicit alignment the model breaks or, worse, mis-maps features.
- **External validity:** results are reproducible on NSL-KDD but the dataset is a frozen 1998-1999 simulation, so reproducing the number says nothing about current traffic.

5. Experimental Results

We trained six classifiers (Logistic Regression, Gaussian Naive Bayes, Linear SVM, Decision Tree, Random Forest, XGBoost) with a fixed seed, 5-fold stratified cross-validation for the same-distribution protocol, and the official KDDTest+ for the honest protocol. We led with MCC and F2 (recall-weighted, because a missed attack is costlier than a false alarm). Results, sorted by test-set MCC:

Model	CV acc	Test acc	Prec	Recall	F1	F2	MCC	ROC-AUC
XGBoost	0.999	0.795	0.969	0.661	0.786	0.706	0.643	0.967
Decision Tree	0.998	0.790	0.959	0.660	0.782	0.704	0.632	0.735
Random Forest	0.999	0.778	0.969	0.630	0.764	0.677	0.619	0.960
Logistic Regression	0.975	0.752	0.915	0.622	0.741	0.665	0.554	0.770
Linear SVM	0.976	0.746	0.914	0.612	0.733	0.656	0.546	0.783
Gaussian NB	0.844	0.557	0.981	0.226	0.368	0.267	0.324	0.801

Table 1. Performance under both protocols, sorted by test-set MCC.

The CV-accuracy column reproduces the source's near-perfect numbers; the test-accuracy and MCC columns show what survives on unseen attacks. The ranking is stable (tree ensembles lead), but the absolute performance is far lower than the headline. The separability curves for the best model are shown in Figure 6.

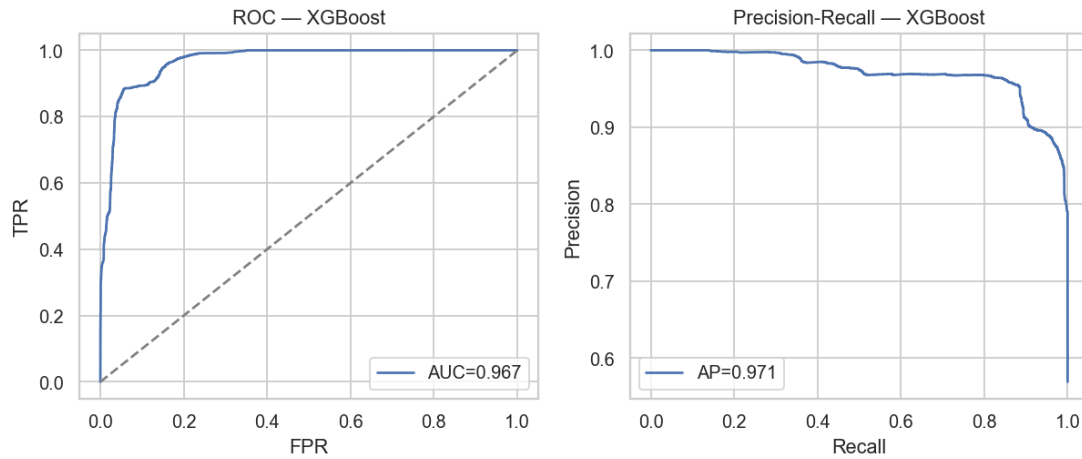


Figure 6. ROC (left) and Precision-Recall (right) for the best model on the official test set. Useful ranking ability remains, but well short of the near-perfect impression accuracy gives.

6. Error Analysis

On the official test set the best model produces 4,355 false negatives (missed attacks, 33.9% of all attacks) and 273 false positives (false alarms). The confusion matrix is shown in Figure 7.

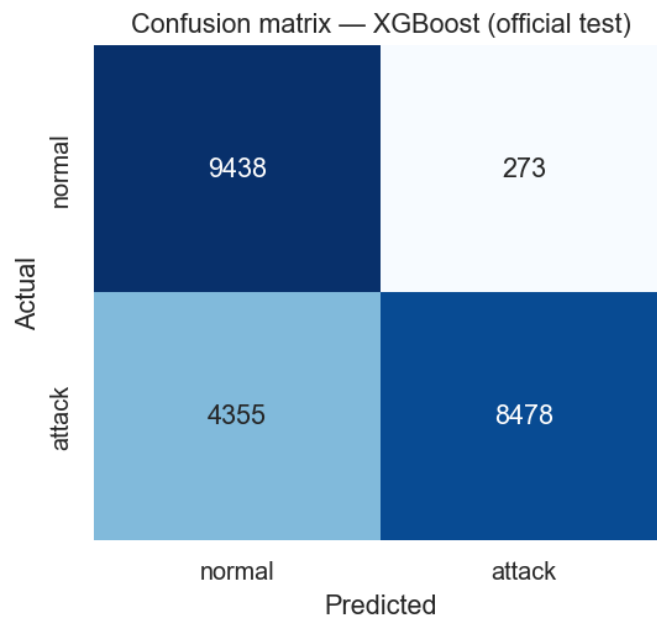


Figure 7. Confusion matrix for the best model (XGBoost) on the official test set. The off-diagonal cells are the operationally important errors analysed below.

- **False negatives** are dominated by R2L/U2R and novel signatures — attacks that look like normal sessions at the packet-statistic level (a single login, a brief privileged shell) and fall on the benign side of a boundary learned from frequent, noisy attacks. Each is an undetected breach.
- **False positives** are unusual but legitimate sessions (bulk transfers, rare services); each costs analyst time.

The trade-off. Because a false negative is a successful intrusion and a false positive is triage time, costs are asymmetric toward recall. That is why we report F2 and MCC and why a deployment would lower the decision threshold and place this model behind correlation

rules rather than trusting a fixed 0.5 cut-off. The source optimises and reports neither.

7. Conclusions

Key findings and lessons

- The ~99.8% accuracy is reproducible but is an artefact of same-distribution evaluation; on the official protocol the same models lose ~20 points and miss most novel and rare attacks.
- For intrusion detection the evaluation protocol matters more than the model: accuracy must give way to MCC, per-class recall and an explicit novel-attack test.
- Always audit features for leakage (difficulty) and dead inputs (num_outbound_cmds), and align categorical encodings across train and test.

Strengths and weaknesses of the source

- **Strengths:** clear scope, runnable code, a sensible attack-category split, and a correct 'Random Forest is best' ordering.
- **Weaknesses:** same-distribution evaluation, a constant feature, no encoding alignment, accuracy-only reporting, no rare-class or novel-attack analysis, and a 25-year-old dataset.

Suggestions for future improvement

- Report on KDDTest+ (or a time-based split) and lead with MCC and PR-AUC.
- Use cost-sensitive learning or resampling to lift R2L/U2R recall.
- Validate on a modern dataset (CIC-IDS2017, UNSW-NB15) before any deployment claim.
- Tune the decision threshold to the recall-heavy cost model of intrusion detection.

8. Summing It Up

Problem: detect malicious network connections among benign traffic. **Source:** the Mamcose NSL-KDD intrusion-detection tutorial. **Dataset:** NSL-KDD, 125,973 train and 22,544 test records, 41 features. **Our study:** faithful reproduction plus a critical battery — a two-protocol comparison (same-distribution CV vs official KDDTest+), per-family and novel-attack recall, a dead-feature and encoding-alignment audit, and a full prevalence-robust metric suite.

Main findings: the ~99.8% accuracy reproduces under cross-validation but falls to 79.5% (MCC 0.6427) on the official test set, with R2L/U2R and novel attacks largely missed. **Were the claims supported?** The narrow 'Random Forest is best' claim, yes; the broad 'near-perfect intrusion detection' claim, no. **Most important insight:** on this benchmark the evaluation protocol, not the algorithm, determines the result. **Recommendation:** useful as a teaching pipeline, but we would not recommend citing its accuracy or deploying it without the corrections above. **Final conclusion:** a methodologically instructive project whose central performance claim is not supported once a realistic evaluation is applied.